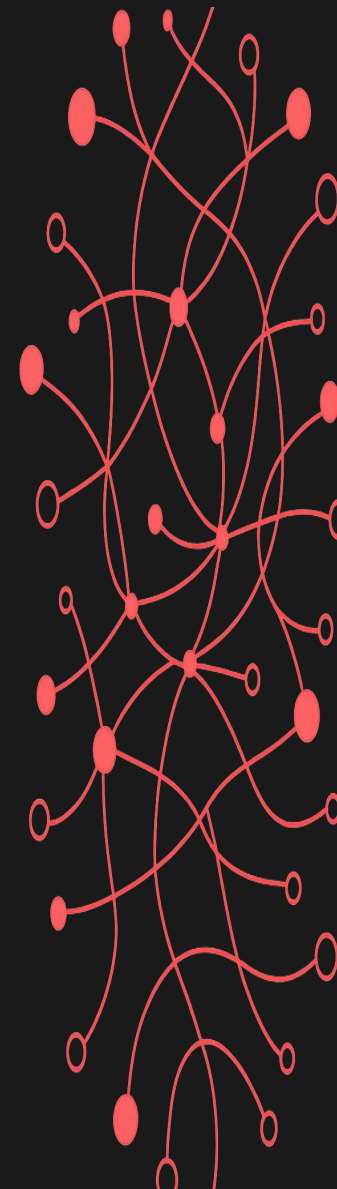


IA Engineering

del prompt a producción

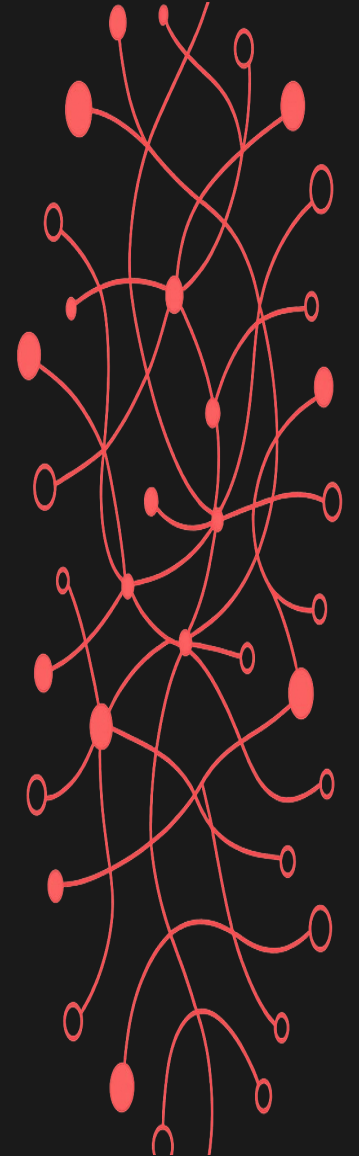
- | 01 Panorama
- | 02 APIs
- | 03 Agents
- | 04 Claude Code



01 Panorama

el estado del arte en 2026

- | LLMs en 1 slide — mental model
- | Cómo funciona — el diagrama
- | El paisaje 2026 — frontier models
- | Reasoning models — el cambio de paradigma
- | Multimodal — más allá del texto
- | Conceptos clave — vocabulario
- | Prompt engineering — los mínimos



LLMs en 1 slide — el modelo mental

Una LLM predice el próximo token. Nada más. Todo lo demás es escala, atención y datos.



↻ *autoregresivo: el token sampleado se agrega al input y se repite el loop*

Lo que tenés que retener:

Token = palabra y media

1 token $\approx$ 0.75 palabras en inglés / $\sim$ 0.5 en español. La unidad de billing.

Context window = lo que el modelo VE

Todo lo que está afuera no existe. Tamaños 2026: típicamente 200K-1M tokens.

Knowledge cutoff

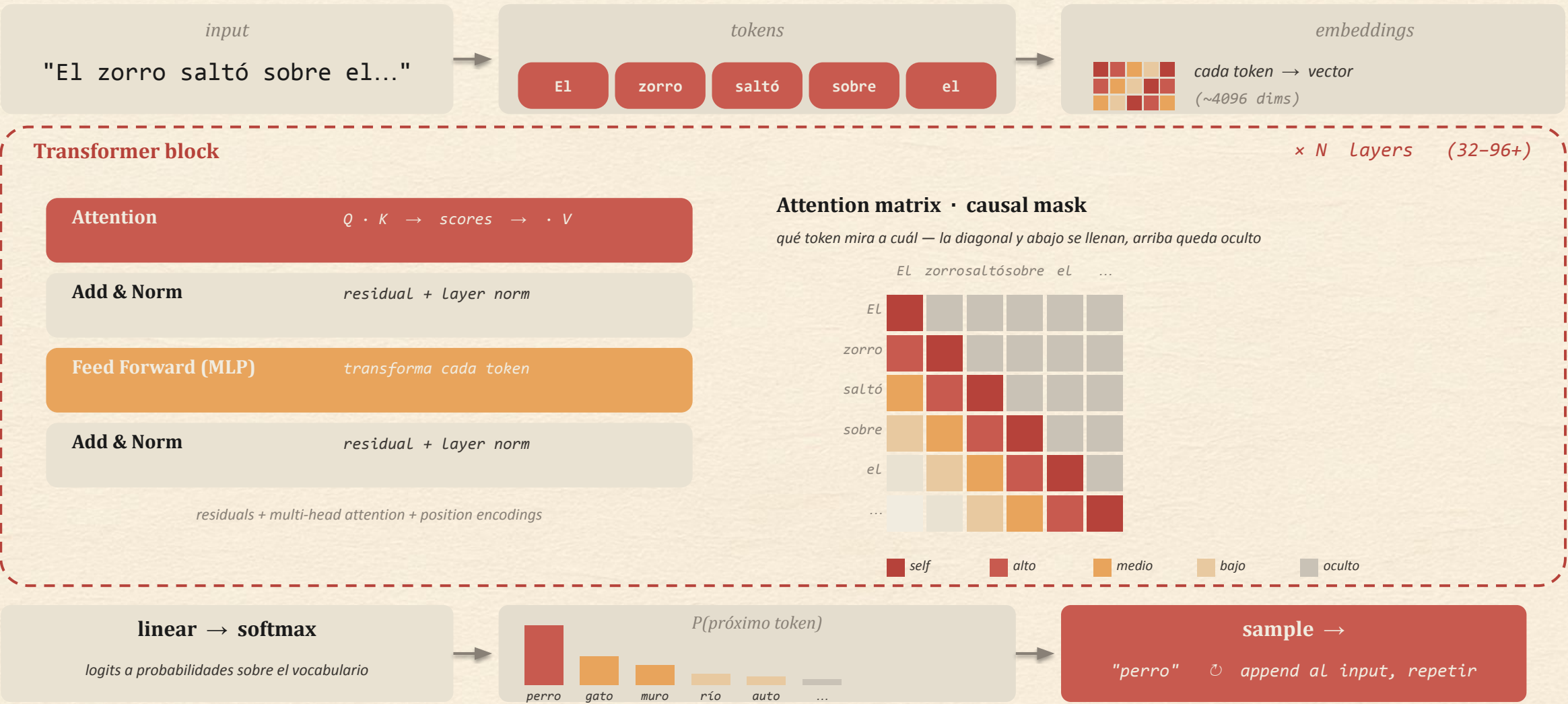
Fecha de los datos de entrenamiento. Después de eso: no sabe (salvo con tools).

Es estadística, no razonamiento puro

Genera lo más probable dado el contexto. Por eso alucina cuando no sabe.

Cómo funciona — el diagrama

Para los curiosos: qué pasa adentro entre 'input' y 'sampled token'. No vamos a profundizar más que esto. Ver: <https://www.youtube.com/watch?v=wjZofJX0v4M>



El paisaje 2026 — quién hace qué

Cuatro grandes familias. Cambian las versiones cada 3-6 meses; el panorama se mueve más lento.

Anthropic	OpenAI	Google	Open source
			
Claude family	GPT / o-series	Gemini family	Llama · Qwen · DeepSeek · Mistral
<i>Opus · Sonnet · Haiku</i>	<i>GPT · o-series · gpt-image</i>	<i>Gemini Pro · Flash · Nano</i>	<i>self-hostable</i>
- Fuerte en código y razonamiento - Context: 200K – 1M tokens - Extended thinking nativo - Computer use · Claude Code - Vertex AI · Bedrock · API	- Ecosistema más amplio - Reasoning models (o-series) - Voice + multimodal pioneros - Response API · Assistants - Azure · API directa	- Context window gigante (1M+) - Multimodal end-to-end nativo - Integración Workspace fuerte - Vertex AI · AI Studio - Modelo on-device (Nano)	- Llama (Meta) — general purpose - Qwen (Alibaba) — multilingual - DeepSeek R1 — reasoning - Mistral — Europa, eficiencia - Hugging Face · Replicate · vLLM

Reasoning models — el cambio de paradigma

El modelo se gasta tokens pensando ANTES de responder. Se llama chain-of-thought, extended thinking, reasoning.

Antes vs ahora:

Modelo base

El modelo predice el próximo token y responde.

Velocidad: instantánea.

Profundidad: limitada por el prompt.

Reasoning model

El modelo escribe 'thinking tokens' (no visibles al user, o resumidos) antes de responder. Itera soluciones, descarta, retoma.

Velocidad: 10x-100x más lento. Profundidad: mucho mayor.

Cuándo conviene · cuándo no:

Sí

Matemática, lógica, multi-step planning.

Sí

Código complejo, refactors no-triviales.

Sí

Análisis profundo de un input grande.

Sí

Tareas donde 'pensar bien' importa más que 'rápido'.

No

Chat conversacional.

No

Extracción simple, clasificación, traducción.

No

Necesitás <1s de latencia.

No

El context ya es chico y la respuesta corta.

Multimodal — más allá del texto

El mismo modelo recibe / produce varias modalidades. En 2026, lo normal son inputs mixtos.



Texto

el default

Conversación, código, structured output. Es lo que vimos hasta ahora.



Vision

imágenes y PDFs

Screenshots, diagramas, fotos de pizarras, PDFs escaneados, frames de video.



Audio

voz in/out

Speech-to-text (transcribir), text-to-speech (responder), conversaciones de voz en tiempo real.



Computer use

el modelo USA la compu

Recibe screenshots, decide click/type/scroll. Anthropic Computer Use, OpenAI Operator.



Code execution

ejecuta código

Sandbox de Python integrado. El modelo escribe y corre código para resolver.



Video / 3D

frames + audio sync

Veo / Sora generan; Gemini ingiere video largo. Aún en evolución rápida.

Conceptos clave — el vocabulario mínimo

Si vas a hablar con cualquier doc de proveedor o paper, vas a ver esto. Vale tenerlo claro.

token

Unidad de texto que ve el modelo. Lo que pagás. 1 tok $\approx$ 0.75 palabras (en).

context window

Cuántos tokens entran en la ventana. 200K-1M en 2026. Más grande $\neq$ siempre mejor.

knowledge cutoff

Fecha de los datos de training. El modelo no sabe nada posterior salvo con tools.

hallucination

Cuando el modelo genera info verosímil pero falsa. Mitigación: grounding + verifier.

temperature / top_p

Controles de sampling. 0 = determinístico, alto = exploratorio / creativo.

system prompt

Las reglas que persisten todo el turno. Lo más fuerte que lee el modelo.

tool use / function calling

El modelo pide que tu código corra una función y le devuelvas el resultado.

structured output

El modelo respeta un JSON Schema. Para producción casi siempre conviene.

prompt caching

Reusar el prefijo del prompt entre requests. -75% a -90% del costo.

eval

Test automatizado contra outputs del modelo. Lo que separa producción de demo.

RAG

Retrieval-augmented generation: traer info externa al prompt en runtime.

agent

LLM en un loop con tools. Decide qué hacer hasta terminar.

Prompt engineering

El arte está sobrevalorado. La disciplina es ingeniería de contexto (la vemos en Módulo 3).

5 reglas que rinden:

1 Sé específico



"mejoré esto"



"reescribo en tono formal, máximo 200 palabras, sin jerga técnica"

2 Da contexto

Quién sos, para quién es, qué constraints aplican. El modelo no sabe nada de tu mundo.

3 Mostrá ejemplos (few-shot)

Cuando importa el formato exacto, 2-3 ejemplos > toda la prosa del mundo.

4 Usá estructura

XML tags para Claude · Markdown para OpenAI · schema cuando podés.

5 Iterá

El primer prompt nunca es el bueno. Probá, fijate qué falla, ajustá.

Un prompt decente, side-by-side:

```
prompt.md

# Rol
Sos un reviewer técnico senior.

# Tarea
Revisá el diff que te paso y devolveme
un JSON con esta forma:

{
  "issues": [
    { "severity": "critical|major|nit",
      "file": "...",
      "line": 42,
      "comment": "..." }
  ],
  "summary": "..."
}

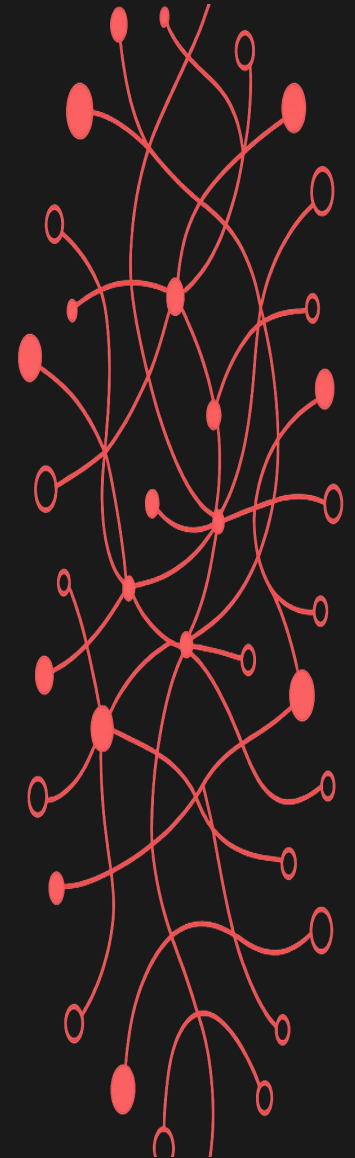
# Reglas
- Bugs reales > style.
- Cubrir tests del cambio.
- Si todo OK: issues=[], summary='LGTM'.

# Diff
<diff>
...
</diff>
```

02 APIs

el contrato entre tu app y el modelo

- | Anatomía de un request
- | Manejo de contexto — Response API vs Chat Completions
- | Tipos de mensaje — system · developer · user
- | Tipos de respuesta — text · json · json_schema
- | Parámetros que mueven la aguja
- | Function calls
- | MCP — un protocolo, mil herramientas
- | RAG / Resources



Anatomía de un request

Qué partes lleva una llamada a una API de LLM.

model

qué LLM usar · ej. gpt-5, claude-opus-4-7

messages[]

system + user + assistant (turns)

tools[]

funciones que el modelo puede invocar

response_format

text · json_object · json_schema

parameters

temperature · max_tokens · top_p · ...

```
POST /v1/responses

{
  "model": "claude-opus-4-7",
  "messages": [
    {"role": "system",
     "content": "Eres un tutor..."},
    {"role": "user",
     "content": "¿Qué es RAG?"}
  ],
  "tools": [{ ... }],
  "response_format": {
    "type": "json_schema",
    "schema": { ... }
  },
  "temperature": 0.7,
  "max_tokens": 1024
}
```


Manejo de contexto · dos APIs, dos enfoques

Chat Completions · stateless



Turn 1: send [user]

client sends 1 message · server returns 1 response



Turn 2: send [user, assistant, user]

client re-envía toda la historia



Turn N: send [u, a, u, a, ... , u]

payload crece cada turno · costo escala con la historia

El estado vive en el cliente.

Response API · stateful



Turn 1: send [user] → response_id_A

el server guarda la conversación



Turn 2: previous_response_id=A + [user]

cliente envía solo el delta · payload constante



Turn N: previous_response_id + [user]

ideal para multi-agentes y tool-use anidado

El estado vive en el servidor.

Tipos de mensaje — los 3 roles

Cada mensaje en una API tiene un rol — y los roles cambian quién manda en la conversación.

system



Las reglas del juego

- Define personalidad, formato, restricciones
- Una sola vez, al inicio
- Lo más fuerte que escucha el modelo

```
{  
  "role": "system",  
  "content":  
    "Sos un tutor de IA.  
    Respondé en español."}
```

developer



Reglas del que integra

- Nuevo en GPT-5 / Response API
- Reglas técnicas que el end-user no debería poder modificar
- Anthropic no tiene este rol — usa system

```
{  
  "role": "developer",  
  "content":  
    "Nunca devuelvas  
    contenido HTML."}
```

user



Lo que pide la persona

- El input efectivo de cada turno
- Puede repetirse N veces
- Lo que un atacante intenta inyectar

```
{  
  "role": "user",  
  "content":  
    "Explicame RAG  
    en 2 oraciones."}
```

Instrucciones — cada proveedor tiene su receta

Mismo concepto, distinta ubicación y nombre. No son intercambiables.



OpenAI

Chat Completions · Response API

Las instrucciones son un mensaje más

role: "system"

Reglas generales del comportamiento.

role: "developer"

Reglas técnicas (solo Response API + GPT-5).

role: "user"

Lo que pide el end-user.

fmt: Markdown — headers + listas

```
"messages": [
  {"role": "system", "content": "..."},
  {"role": "developer", "content": "..."},
  {"role": "user", "content": "..."}
]
```

Tip: Reglas hard en developer, estilo en system.



Anthropic

Messages API · Claude Code SDK

Las instrucciones son un campo top-level

system: "..."

Campo separado del array messages.

role: "user"

Solo turns conversacionales adentro.

role: "assistant"

Respuestas previas del modelo.

fmt: XML tags — <example>, <rules>, <context>

```
"system": "Sos un tutor de IA...",
"messages": [
  {"role": "user", "content": "..."},
  {"role": "assistant", "content": "..."}
]
```

Tip: system breve y directo — Claude lo respeta mucho.



Google · Vertex AI

Gemini API

Similar a Anthropic, distinta nomenclatura

system_instruction

Top-level, separado de contents.

role: "user"

Turns del usuario.

role: "model"

Respuestas previas (¡no assistant!).

fmt: Markdown estructurado + few-shot examples

```
"system_instruction": {
  "parts": [{"text": "..."}]
},
"contents": [
  {"role": "user", "parts": [...]},
  {"role": "model", "parts": [...]}
]
```

Tip: Pocas instrucciones, ejemplos dentro de contents.

Tipos de respuesta — qué le pedís al modelo

Cuanto más estructurada querés la salida, más restricciones le ponés al modelo.

text

natural language

- Más libre, más expresiva
- Lo que ves en un chat
- Ideal para storytelling, escritura, explicaciones

request:

```
"text": {
  "format": {
    "type": "text"
  }
}
```

response:

```
"Claro! RAG significa
Retrieval-Augmented
Generation..."
```

json_object

structured, but free-form

- El modelo devuelve JSON válido
- Sin schema fijo: el modelo decide los campos
- Útil cuando el shape es flexible

request:

```
"text": {
  "format": {
    "type": "json_object"
  }
}
```

response:

```
{
  "tema": "RAG",
  "resumen": "..."
```

json_schema

strict, schema-validated

- Definís el schema (Pydantic, Zod)
- El modelo está obligado a cumplirlo
- Best practice para producción

request:

```
"text": {
  "format": {
    "type": "json_schema",
    "schema": {...}
  }
}
```

response:

```
{
  "tema": "RAG",
  "score": 9.2,
  "fuentes": [...]
```

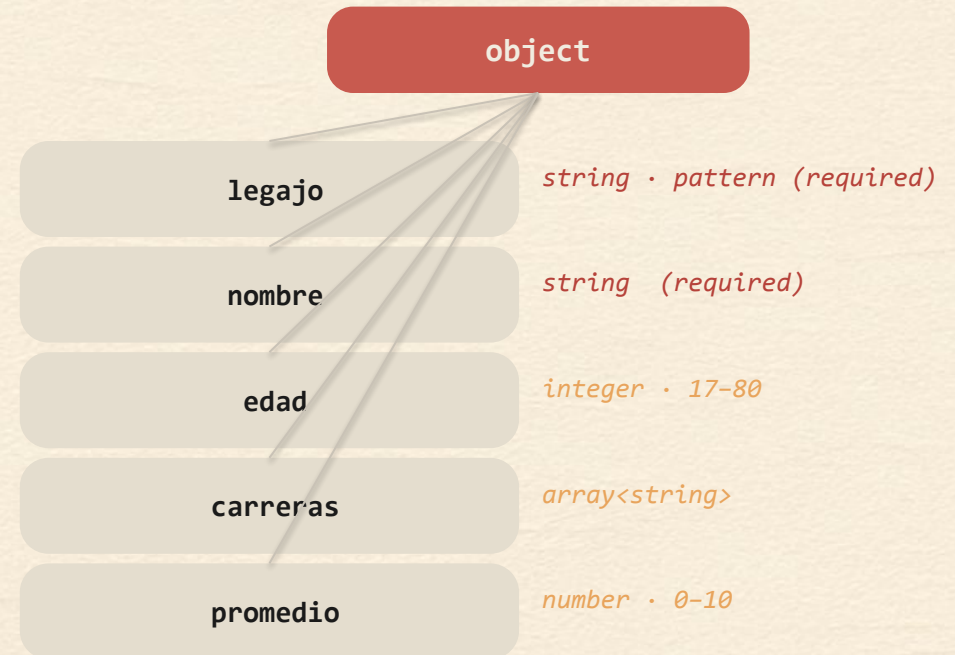
Anatomía de un JSON Schema

Es la piedra angular: `response_format`, `function args` y `MCP tools` — todos usan JSON Schema.

```
alumno.schema.json

{
  "type": "object",
  "properties": {
    "legajo": { "type": "string",
               "pattern": "[0-9]{5}$" },
    "nombre": { "type": "string" },
    "edad": { "type": "integer",
              "minimum": 17, "maximum": 80 },
    "carreras": {
      "type": "array",
      "items": { "type": "string" }
    },
    "promedio": { "type": "number",
                  "minimum": 0, "maximum": 10 }
  },
  "required": ["legajo", "nombre"],
  "additionalProperties": false
}
```

Lo que el modelo entiende:



El mismo schema declara: `response_format` · `function args` · `MCP tool inputs` · `Pydantic` / `Zod` compilan a esto.

Parámetros — los que mueven la aguja

Lo que cambia el comportamiento del modelo más allá del prompt.



temperature

0.0 ↔ 2.0

Cuán creativo / aleatorio. Bajo = determinístico, alto = exploratorio.



top_p

0.0 ↔ 1.0

Nucleus sampling. Filtra tokens hasta acumular probabilidad p.



max_tokens

1 ↔ context

Cuántos tokens puede generar la respuesta. Cap duro.



stop

string[]

Sequences que cortan la generación si aparecen.



reasoning_effort

Low / med / high

En modelos de razonamiento: cuánto puede pensar antes de responder.



seed

int

Reproducibilidad — misma seed + mismo prompt ≈ misma respuesta.



tool_choice

auto / required / none

Política sobre cuándo invocar tools. 'required' fuerza tool use.



parallel_tool_calls

bool

Permite que el modelo invoque varias tools en un solo turn.

Pricing y caching — la moneda son los tokens

Cada response trae cuánto consumiste. Cachear el prefijo es el gran ahorro — cada proveedor lo hace distinto.

Input tokens

Todo lo que mandás:
system + messages + tools

Output tokens

Lo que devuelve el modelo.
Más caros (~2-5×) que input.

Cached tokens

Prefijos repetidos.
Descuento de ~75-90%.

Cada response trae tu consumo:

```
response.usage
{
  "input_tokens": 1480,
  "output_tokens": 312,
  "cache_read_tokens": 1024,
  "cache_creation_tokens": 0,
  "total_tokens": 2816
}
```

Estrategias de caching — el mismo objetivo, tres mecánicas:



OpenAI

Implícito — automático

- Se activa solo si el prefijo del request $\geq \sim 1024$ tokens
- Vos no marcás nada — el sistema busca match
- TTL ~5–10 min (hasta ~1h en horas tranquilas)
- Best practice: contenido estático arriba, dinámico abajo



Anthropic

Explícito — vos marcás

- Agregás `cache_control: {"type": "ephemeral"}` en el bloque
- Hasta 4 breakpoints por request
- Cobra una vez al crear, después es 90% más barato
- Ideal: system prompts, RAG, herramientas largas



Google · Vertex AI

Explícito — generás y attacheás

- Creás un `CachedContent` vía API (POST `/cachedContents`)
- Recibís un nombre tipo `cachedContents/abc123`
- Lo referenciás en cada request: `cached_content: "..."`
- TTL configurable · vos pagás el storage mientras vive

Function calls — el modelo invoca tu código

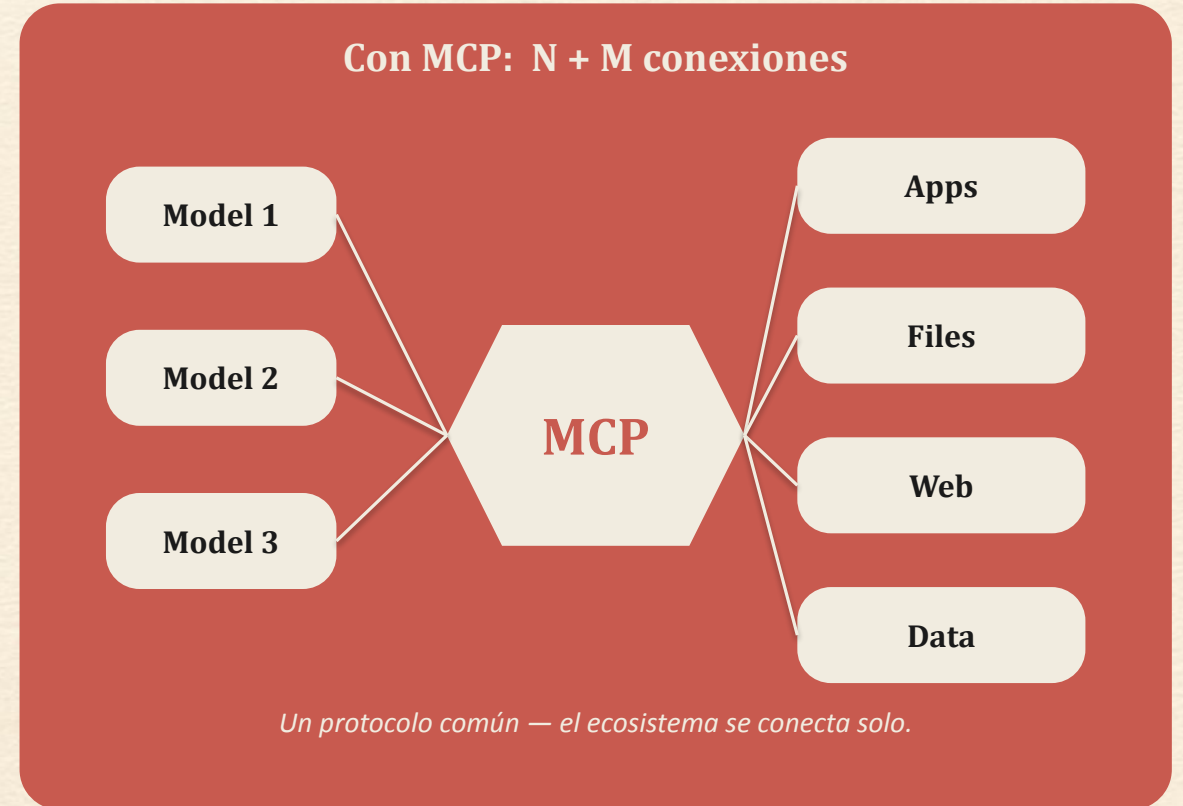
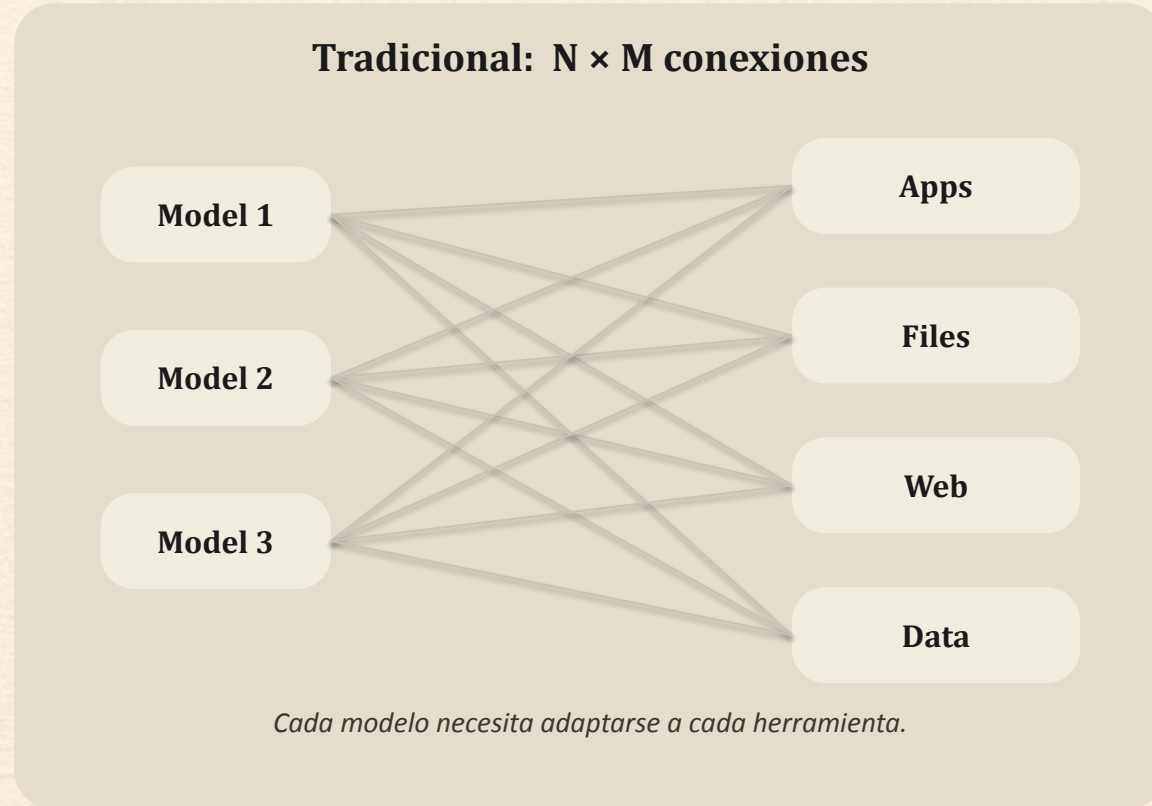
El modelo no ejecuta nada. Solo decide qué función llamar; vos la corrés y le devolvés el resultado.



	Anthropic Messages API	OpenAI Chat Completions	OpenAI Response API
nombre del param.	tools	tools	tools
resultado del modelo	tool_use block (en content[])	tool_calls	tool_calls
devolver resultado	tool_result block (en user msg)	role=tool	function_call_output
paralelo	✓ por defecto	parallel_tool_calls	parallel_tool_calls

MCP — el estándar abierto para integrar herramientas

Model Context Protocol (Anthropic, 2024) — un protocolo, mil herramientas.



Los 3 primitivos que expone un MCP server:

Tools · funciones que el modelo puede invocar (POST, side-effects)

Resources · datos read-only que el modelo puede consultar (files, schemas)

Prompts · templates reutilizables que el server propone al cliente

RAG / Resources — más allá del entrenamiento

Dos formas de darle al modelo acceso a información que no está en su entrenamiento.

RAG — Retrieval-Augmented Generation



Documents



Chunks



Embeddings



Vector DB



Retrieval



Augmented prompt

MCP Resources — alternativa moderna

Sin pipeline propio

El MCP server expone los datos directamente.

Read-only

El modelo consulta · no escribe (eso es Tools).

Versionable, governable

Cada Resource tiene URI, schema, permisos.

Casos típicos

Repositorios de docs · schemas de bases · API specs · changelogs.

Convive con RAG

No es uno u otro — Resources puede alimentar el retrieval.

RAG en acción — un walkthrough

Bot de una tienda de zapatillas. El usuario ya preguntó sobre devoluciones; ahora hace una pregunta de seguimiento.

Contextualización

1

rewrite con la conversación previa

```
"¿Y si pagué con débito?" ← query corta y ambigua
history: [user: ¿Cuál es la política de devoluciones?] [assistant: Aceptamos...]
→ rewrite: "¿Hay devolución para compras pagadas con débito?"
```

Embedding

2

vectorizar la query reescrita

```
embed_query("¿Hay devolución para compras pagadas con débito?")
→ [ 0.123, -0.045, 0.890, -0.221, ..., 0.234 ] (1536 dims)
```

Búsqueda en el vector DB

3

top-k por similitud coseno

```
score 0.91 · chunk #214 · "Devoluciones aceptan débito y crédito."
score 0.87 · chunk #142 · "El plazo de devolución es de 30 días."
score 0.74 · chunk #389 · "Devoluciones requieren ticket original."
```

Augmented prompt

4

armar el prompt final

```
system: Respondé SOLO con la info dada. Si no hay info, decí 'no sé'.
context: [#214] [#142] [#389]
user:    ¿Hay devolución para compras pagadas con débito?
```

LLM responde

5

con la info del context

```
"Sí, aceptamos devoluciones con débito hasta 30 días, presentando el ticket original."
```

Citas (opcional)

6

el modelo apunta a las fuentes

```
fuentes: [chunk #214] [chunk #142] [chunk #389]
→ útil para auditar y mostrar al user de dónde salió la respuesta.
```

RAG — cuándo NO usarlo

RAG es búsqueda, no es lectura. Si tu pregunta necesita TODO el documento, RAG va a fallar.



Tareas con orden / cronología

Resumir un changelog en orden, contar la historia de un personaje, auditar logs cronológicamente.

- ✗ RAG devuelve chunks por similitud, no por orden.
- ✓ Mejor: leer el doc completo o usar la herramienta nativa (logs ordenados, base de datos).



Resúmenes globales

“Resumime los 300 reviews de este producto.” “¿Cuáles son los temas principales del libro?”

- ✗ RAG trae top-k chunks. Pierde el bosque.
- ✓ Mejor: map-reduce sobre todos los chunks, o un modelo con 1M context.



Razonamiento cross-chunk

“Compará la cláusula 3 con la cláusula 17 y decime si se contradicen.”

- ✗ Si las 2 cláusulas no caen juntas en el top-k, el modelo no las ve.
- ✓ Mejor: agentic retrieval (el modelo decide qué buscar) o lectura completa.



Información que cambia rápido

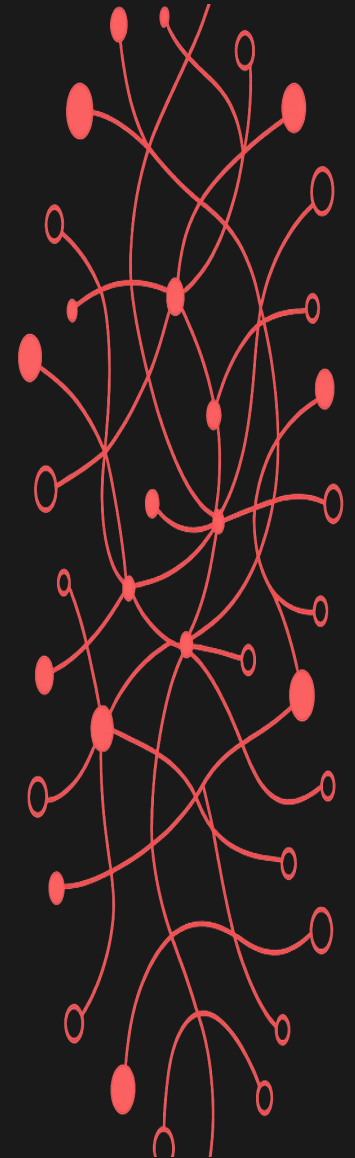
Stock en tiempo real, precios actuales, status de un pedido.

- ✗ El índice vectorial está siempre desactualizado.
- ✓ Mejor: function call directo a la API de datos en vivo.

03 Agents

del chat al colaborador autónomo

- | ¿Qué es un agente? — chat vs workflow vs agent
- | ¿Cuándo usar un agente — y cuándo no?
- | Turn · Tools · Tools dinámicas · Sandbox
- | El loop walked-through · Stop conditions & guardrails
- | Context lifecycle + Context engineering
- | Skills — procedimientos guardados
- | Subagentes — básico + patrones



¿Qué es un agente?

No todo lo que llama una LLM es un agente. Compará los tres patrones.

Chat



Conversacional

- Humano pregunta · modelo responde
- Sin tools, sin acciones
- Sin estado interno
- Cada turn lo decide el humano

El modelo habla.

Workflow

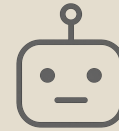


Cadena predefinida

- Pasos fijos, decididos por vos
- El LLM aparece en ciertos nodos
- Determinista en su estructura
- Predecible y testeable

Vos decidís el camino.

Agent



Loop autónomo

- El modelo decide qué hacer y cuándo
- Llama tools en un loop
- Mantiene estado entre turns
- Termina cuando cree que terminó

El modelo decide el camino.

¿Cuándo usar un agente — y cuándo no?

Decisión práctica: un agente es más complejo y más caro que un workflow. Vale la pena solo si el problema lo requiere.

Usá un agente cuando...

- **El espacio de soluciones es abierto**

No podés enumerar los pasos por adelantado. Ej: investigar bugs, planear viajes.

- **Hay decisiones intermedias**

El siguiente paso depende del resultado del anterior. Ej: leer un archivo y decidir cuál tocar.

- **Necesitás tools reales**

Llamadas a APIs, ejecución de código, lectura de archivos, búsqueda.

- **Ahorra trabajo a un humano**

Tareas largas, repetitivas, con muchos pasos verificables.

- **Tenés cómo verificar el output**

Tests, oracles, schemas — si no, no sabés si el agente terminó bien.

Quedate con un workflow cuando...

- **Los pasos son fijos**

Si podés escribir el pseudocódigo de la solución, no necesitás un agente.

- **Una sola LLM call alcanza**

Clasificación, traducción, resumen, extracción estructurada → un prompt.

- **Latencia es crítica**

Un agente puede hacer 5-20 calls. Si necesitás <1s, no funciona.

- **No hay tools que aporten**

Si todo se resuelve con texto, no estás aprovechando lo que un agente es.

- **Costo descontrolado**

Cada turn paga tokens. Sin guardrails de max_turns, te fundís la plata.

Turn — la unidad básica del loop

Un turn es una llamada al LLM más, si aplica, la ejecución de las tools que pida. El loop son muchos turns.

Anatomía de un turn:

- 1 Build request** *messages[] + tools[] + params*
- 2 POST /v1/messages** *→ LLM API*
- 3 Response** *text ó tool_use[] ó ambos*
- 4 ¿Hay tool_use?** *no → fin del loop · sí → ejecutar*
- 5 Ejecutar tool(s)** *tu backend corre la función*
- 6 Append tool_result** *al messages[] · vuelta al paso 1*

En código:

```
agent_loop.py

messages = [system, user_input]

while True:
    response = llm.create(
        messages=messages,
        tools=TOOLS,
    )
    # ← cada vuelta = 1 turn

    if response.stop_reason == 'end_turn':
        return response.text
    # fin del loop

    # response contiene tool_use[]
    for tool_call in response.tool_uses:
        result = execute(tool_call)
        messages.append(tool_result(result))
```

Tools dinámicas — el agente arma sus propias

Las tools no tienen que estar todas predefinidas. El agente puede generar las suyas en runtime.

Code execution

una tool universal

Tu harness expone una sola tool — `run\_python` o `bash` — y el modelo escribe código sobre la marcha.

Una sola tool, infinitas funcionalidades.

Ej: para extraer la columna 3 de un CSV, escribe pandas en vivo. Para parsear un log, escribe regex.

```
run_python(code="""
import pandas as pd
df = pd.read_csv('data.csv')
print(df['nombre'].unique())
""")
```

Tool composition

chainer lo que ya hay

El modelo combina tools existentes en patrones nuevos. Sin código nuevo — solo decide qué encadenar.

Para “exportar mis contactos a PDF” el agente:

1. query_db(tabla=contactos) → 2. render_pdf(data=...) → 3. send_email(attach=...)

```
# pseudo-flow
rows = query_db("contactos")
pdf = render_pdf(rows)
send_email(to=user, attach=pdf)
```

Skills

procedimientos guardados → veremos en detalle

Vos registrás un procedimiento (SKILL.md). El agente lo descubre on-demand y lo invoca como tool.

El skill describe CUÁNDO usarlo y CÓMO. Sumás capacidades sin tocar el código del harness.

```
# ~/.claude/skills/export-contacts.md
name: export-contacts
description: "Exporta contactos del CRM filtrados por país a PDF."
steps:
  - query_db ...
```

tradeoff: *más expresividad ↔ más sandbox y observabilidad. Tools dinámicas son potentes — y superficie de ataque.*

Skills — procedimientos guardados, no funciones

Una skill es un procedimiento en markdown. El agente la descubre on-demand y la usa como si fuera una tool.

```
~/.claude/skills/triage-bug.md

---
name: triage-bug
description: |
  Triage an incoming bug report.
  USE WHEN: a user describes a defect.
  DON'T USE WHEN: it's a feature request.
allowed-tools:
  - Bash(git log:*)
  - Read(./logs/**)
  - Read(./src/**)
args:
  - name: report
    description: the user's bug report
---

## Procedure

1. Read the report. Extract: component,
   reproduce steps, expected vs actual.
2. `git log --since='7 days ago' -- <component>`
3. Read relevant logs in ./logs/.
4. Cross-reference recent commits with the
   error message.
5. Output: a triage note with the most likely
   commit and a confidence score.
```

Cómo se diferencia de una tool:

- 1 Es un archivo, no código**
Vivís el procedimiento en markdown. Lo edita un humano o el propio agente.
- 2 Progressive disclosure**
El agente lee la skill SOLO cuando es relevante. No infla el system prompt.
- 3 Composable**
Una skill puede invocar otras skills, o tools, o subagents.
- 4 Tiene metadata**
frontmatter con name, description, allowed-tools, args. La description es lo que el modelo lee.
- 5 Vive en disco / repo**
Versionable en git. Compartibles con el equipo. Skill packs / plugins.

Sandbox — ejecutar código del modelo sin morir

Una tool de code-exec sin aislamiento es una bomba. Un `rm -rf`, un `curl` con tus secrets, y se acabó.

Capas que tenés que aislar:



Filesystem

*scratch dir efímero · resto read-only ·
sin /etc, sin ~/.ssh*



Network

*deny por default · allowlist de
dominios necesarios*



Compute

*CPU + RAM caps · timeout duro
(segundos, no minutos)*



Identidad

*sin env vars de prod · sin tokens · sin
perfil cloud*



Output

*tope al tamaño de stdout · sin
streaming de archivos grandes*

Cómo se implementa (de menos a más fuerte):

Docker container

rápido · ampliamente soportado

OK para la mayoría de los casos · cuidado con escape.

microVM (Firecracker, Kata)

kernel aislado · start ~150ms

Es lo que usa Anthropic Code Execution, AWS Lambda, Fly Machines.

Managed cloud sandboxes

no te ocupás de la infra

Cloud Run sandboxes · Bedrock AgentCore · E2B · Modal.

Browser sandbox

para tools que tocan la web / la UI

Claude in Chrome · Computer Use corre en una VM controlada.

principios: *ephemeral · least privilege · timeout-killed · output capped · reproducible / borrrable cada turn*

El agent loop — caso práctico

Tarea: "Arreglá el test que falla en test_user.py". El agente tiene tools: `read_file`, `edit_file`, `run_tests`.

turn 1	
1	read_file
	<code>tool_use: read_file(path="test_user.py")</code> → contenido del test (12 líneas)
turn 2	
2	read_file
	<code>tool_use: read_file(path="user.py")</code> → contenido de user.py (48 líneas)
turn 3	
3	run_tests
	<code>tool_use: run_tests(file="test_user.py")</code> → FAIL: test_age expected 25 got 26 (off-by-one)
turn 4	
4	edit_file
	<code>tool_use: edit_file("user.py", old="age + 1", new="age")</code> → OK · diff aplicado
turn 5	
5	run_tests
	<code>tool_use: run_tests(file="test_user.py")</code> → PASS · 1/1 ok
turn 6	
6	respuesta
	<code>stop_reason: end_turn</code> → "Arreglé el off-by-one en user.calculate_age."

6 turns, 5 tool_use, 1 end_turn. Cada turn el `messages[]` del contexto crece con `tool_use` + `tool_result`.

Stop conditions & guardrails — controlar el loop

Sin esto, un agente puede correr para siempre. Y vaciarte la cuenta haciéndolo.

Cómo termina un turn (stop_reason):

end_turn	<i>el modelo terminó — fin del loop</i>
tool_use	<i>pidió tools — el loop sigue</i>
max_tokens	<i>agotó max_tokens de output</i>
stop_sequence	<i>matcheó una stop sequence configurada</i>
refusal	<i>el modelo se negó (policy)</i>
error	<i>el provider devolvió error</i>

Tus guardrails (tu código):

max_turns

Hard cap. Si el loop pasa N turns, abortar. CRÍTICO.

Budget total

Límite de tokens o \$ por sesión. Para si lo excede.

Timeout

Tope de wall-clock. Útil para tools lentas (web, DB).

Allow / deny list

Tools que el agente PUEDE usar. Y las que no.

Confirmación humana

Antes de side-effects destructivos: pedí OK al user.

Validación de outputs

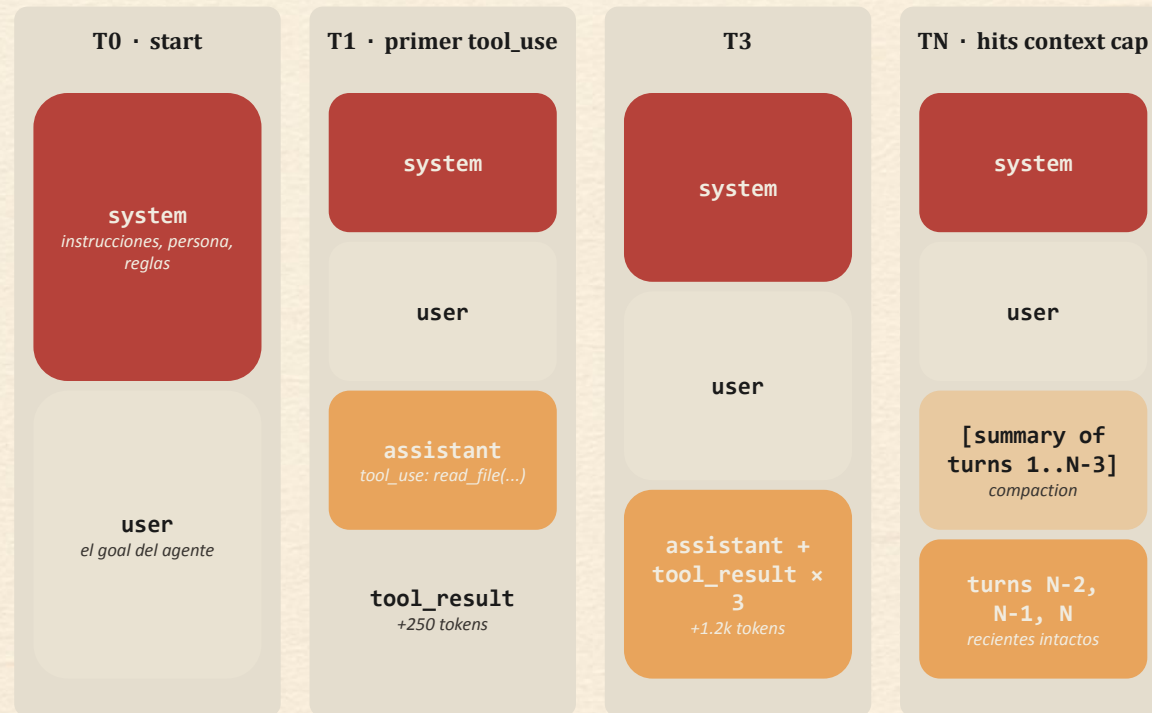
Verificá el tool_result antes de pasarlo al siguiente turn.

anti-pattern: `while True: response = llm.create(...)` → *Loop infinito + factura imposible. Siempre con max_turns.*

Context — cómo se va manejando turn a turn

El contexto NO es estático: crece cada turn (tool calls + results) y eventualmente hay que podarlo.

Qué hay en messages[] turn a turn:



Estrategias para que no explote:

- 1 Tool result pruning**
Después de procesar el resultado, lo reemplazás por un summary corto.
- 2 Compaction**
Cada N turns, pedís al LLM que resuma turns viejos. Bajás 80% el tamaño.
- 3 Subagentes**
Delegás trabajo a un agente con su propio context window. Recibís solo el resumen.
- 4 Tool de memory**
Una tool que guarda hechos clave fuera del context (file system, kv-store).
- 5 Hard caps**
max_turns, max_tokens, budget total — para no fundirte la plata.

Context engineering — la disciplina

Lifecycle es la mecánica. Context engineering es la decisión de QUÉ poner — y dónde.

Context engineering : LLM apps :: database design : backends

lo que ponés y dónde lo ponés define la calidad del output más que el modelo que elegís.

Decisiones de diseño:

Estático vs dinámico

Estático va en system, una vez (y se cachea). Dinámico se trae por turn (RAG, tool results).

Dónde poner los ejemplos

Few-shot al final del system. En system muy largo se diluyen — usá XML tags / headers para anclar.

Cómo pasar tool results

Crudo vs procesado. Una respuesta de 10k tokens de una API: ¿se la pasás entera o le hacés un resumen?

Scratchpad como memoria

Una tool 'remember(key, value)' deja al agente escribir hechos clave fuera del context window.

Estructura del prompt

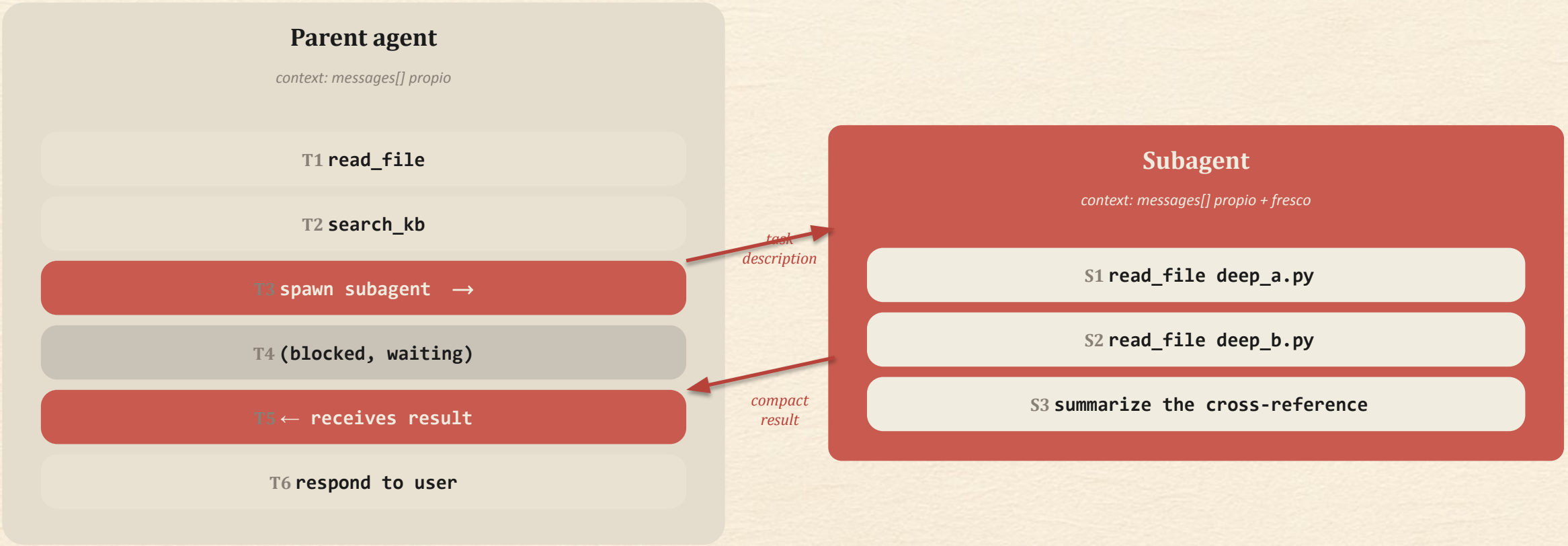
Markdown vs XML vs JSON. Anthropic: XML tags. OpenAI: Markdown. Probá.

Failure modes

Olvido (info enterrada), distracción (demasiados ejemplos), confusión (instrucciones contradictorias).

Subagentes — cuándo se invocan, cómo retornan

Un subagente es otro agente que el parent invoca como si fuera una tool. Tiene SU PROPIO context window.



Cuándo invocarlo

Tarea grande / paralela / fuera del scope del context del parent.

Qué le pasás

Solo la task description. NO le pasás todo tu context (sería inútil).

Qué recibís

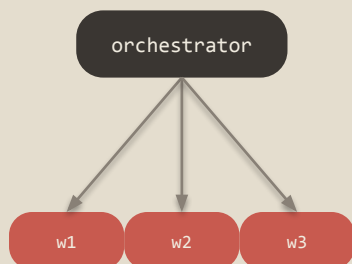
Un resumen compacto. El subagente gastó tokens, pero TU context no creció.

Subagentes — patrones de composición

Llamar a un subagente es fácil. Lo difícil es cuándo y cómo organizar varios.

Orchestrator + workers

un planificador delega N tareas

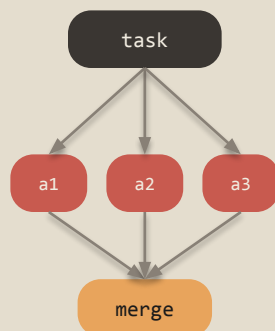


Tarea grande, descomponible.
El orquestador decide el plan y lanza un worker por subtarea.

Ej: research de mercado — un worker por competidor.

Parallel fan-out

el mismo trabajo, N veces

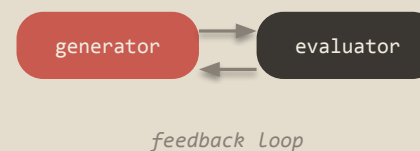


Tarea que se beneficia de votación o exploración paralela.

Ej: 5 generadores compiten, el mejor output gana.

Evaluator-optimizer

uno crea, otro juzga



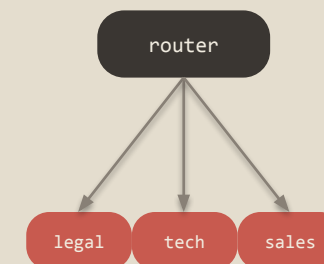
feedback Loop

Calidad mejora con feedback.
El generator produce, el evaluador critica, el generator itera.

Ej: pipeline tipo Generator/ Evaluator.

Specialist routing

un router elige el especialista



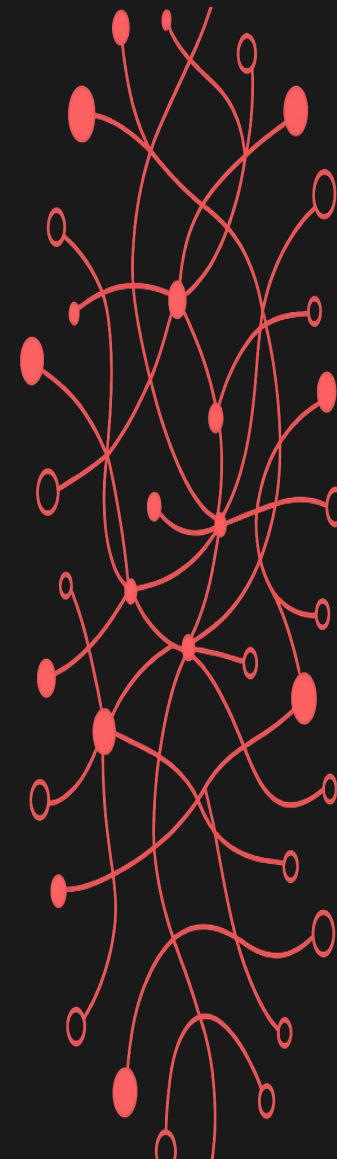
Dominio mixto, un solo agente no rinde.

Ej: legal vs técnico vs ventas
→ router decide a quién mandar.

04 Claude Code

el agente que vive en tu terminal

- | ¿Qué es Claude Code? · dónde corre
- | Setup — tu primer claude en 60 segundos
- | CLAUDE.md + niveles de configuración
- | Plan mode — pensá antes de tocar
- | Subagents / Agent Teams
- | Skills · Hooks · MCP — extender el agente
- | Permissions & Auto Mode
- | Cheat sheets: slash commands + tools
- | Continuará → clase de SDD



¿Qué es Claude Code?

Un agente que vive donde escribís código. Lee el repo, corre tests, abre PRs.



Claude Code

Coding agent terminal-native, hecho por Anthropic.

- **Agentic, no autocompletado**

Decide qué hacer, llama tools, itera hasta terminar la tarea.

- **Funciona con tu repo**

Lee archivos, corre tests, ejecuta bash, edita en lugar.

- **Modelo subyacente**

Claude Opus / Sonnet / Haiku — elegís según el trabajo.

- **Configurable**

CLAUDE.md, skills, hooks, MCP, permisos.

Dónde corre:



Terminal



VS Code / JetBrains



Web (claude.ai/code)



Desktop (Mac / Win)



Mobile (iOS / Android)



Slack

Tu primer claude — en 60 segundos

Instalás, te logueás, te paras en tu repo y le decís qué querés.

1. Install

npm vs homebrew vs curl



```
# npm — la forma más común
npm install -g @anthropic-ai/claude-code

# macOS — alternativa
brew install claude-code

# Linux / Mac — script
curl -fsSL https://claude.ai/install | sh
```

2. Auth + open project

claude login + cd al repo



```
$ claude login
→ abre el browser para OAuth
→ vincula tu cuenta Claude

$ cd ~/code/mi-proyecto
$ claude
→ Claude Code v2.x
→ Opus 4.7 (1M context)
→ ~/code/mi-proyecto
```

3. Decile qué querés

lenguaje natural, no flags



```
> Read README.md y explicame
    qué hace este proyecto.

> Hay un bug en /api/users.
    Cuando el legajo tiene
    ceros adelante, falla.
    Encontralo y arreglalo.

(shift+tab → plan mode)
```


CLAUDE.md — el cerebro del proyecto

Un archivo markdown en la raíz del repo. Claude lo lee primero en cada sesión.

Coding conventions

Las reglas no escritas que tu equipo ya conoce.

How to run things

Build, test, lint — los comandos que tipeás sin pensar.

Architecture map

Dónde vive cada cosa, quién habla con quién.

Landmines

Files que no se tocan, patterns que no se usan.

Project glossary

Términos del dominio. Claude no nació en tu empresa.

```
CLAUDE.md

# Mi-Proyecto
Backend de e-commerce. Python 3.12 + FastAPI.

## Layout
src/api/      ← endpoints HTTP
src/services/ ← lógica de negocio
src/repos/    ← acceso a DB (NUNCA llamar desde api/)
tests/        ← pytest, target 80% coverage

## Commands
uv run pytest           # tests
uv run ruff check .     # lint
uv run mypy src/        # types
docker compose up -d db # DB local

## Rules
- Timestamps en ISO 8601 con sufijo 'Z'
- Money: usar Decimal, NUNCA float
- Legajos: string, NO int (pueden empezar con 0)
- No tocar src/legacy/ — está siendo migrado

## Incident playbook
1. uv run pytest -x -v
2. Revisar logs en ./logs/api.log
3. Si hay 5xx, mirar Sentry primero
```

Niveles de configuración — qué pisa qué

CLAUDE.md, skills, settings, MCP — todos viven en una cascada bien definida.

4 niveles (de más general a más específico):

User global

~/.claude/

Todos tus proyectos. Tus preferencias personales.

CLAUDE.md · settings.json · skills/ · agents/

Project / repo

./CLAUDE.md + ~/.claude/

Compartido con el equipo (committed en git).

CLAUDE.md · .mcp.json · .claude/{settings,skills,agents}

Subfolder

./apps/api/CLAUDE.md

Solo aplica cuando Claude trabaja en esa carpeta.

Útil en monorepos con stacks distintos por carpeta.

Local override

*.local.md + *.local.json

Gitignored — tu capa personal en este repo.

CLAUDE.local.md · settings.local.json

Layout típico en disco:

```
filesystem

~/.claude/
├── CLAUDE.md
├── settings.json
├── skills/
└── agents/

# USER GLOBAL
# tus prefs en todos los repos
# tus permisos / hooks / env
# tus skills globales
# tus subagentes globales

mi-proyecto/
├── CLAUDE.md
├── CLAUDE.local.md
├── .mcp.json
├── .claude/
│   ├── settings.json
│   ├── settings.local.json
│   ├── skills/
│   ├── agents/
│   └── plugins/
└── apps/
    └── api/
        └── CLAUDE.md

# PROJECT / REPO
# ← committed, todo el equipo
# ← LOCAL OVERRIDE (gitignored)
# MCP servers del repo

# project-level perms / hooks
# ← LOCAL OVERRIDE
# /ship, /triage, etc.
# code-reviewer, ...
# plugin packs instalados

# SUBFOLDER — solo en apps/api
```

Plan mode — pensá antes de tocar

Discutir un plan es barato. Refactorear medio repo y darse cuenta que estaba mal, no.

Cómo se usa:

- **Shift + Tab**

Activa plan mode. Claude pasa a read-only — puede leer, no puede escribir ni ejecutar.

- **Hablale como a un colega**

"No codees todavía, planeá." · "Caminame por tu plan antes de tocar nada."

- **Dejá que Claude te entreviste**

Los mejores planes salen cuando Claude te hace preguntas antes.

- **Aprobás → ejecuta**

Una vez de acuerdo, Shift+Tab de nuevo y Claude ejecuta el plan.

- **Iterás barato**

Disentir de un plan es texto. Disentir de un PR ya hecho es horas de trabajo.

```
Terminal — plan mode

Claude Code v2.x · Opus 4.7
~/code/mi-proyecto

• plan mode on (shift+tab to cycle)

> No codees todavía. Necesito agregar
  retries con backoff exponencial al
  cliente de billing. Decime tu plan.

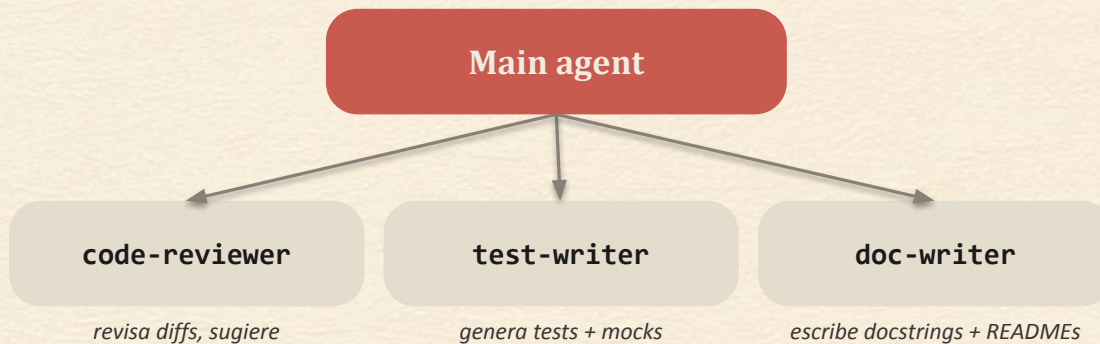
Plan:
1. Wrappear http_client en BillingClient
2. Agregar tenacity con retry on
   ConnectionError + 5xx
3. wait_exponential(min=1, max=30)
4. Modificar 3 call-sites en
   services/payment.py
5. Tests: mockear http, verificar
   que reintente 3 veces

¿Pregunta: querés timeout global o
por-request?
```


Subagents / Agent Teams — delegá a especialistas

Definís sub-agentes con su propio system prompt y tools. El main agent los invoca como tool.

Cómo se ve en runtime:



Cada subagente tiene su propio context window.

El main solo ve el resultado compacto. Resultado: más trabajo, menos context bloat.

```
.claude/agents/code-reviewer.md

---
name: code-reviewer
description: |
  Reviews code changes for bugs,
  style, and architectural fit.
  USE WHEN: a diff is ready to review.
model: claude-sonnet-4-6
allowed-tools:
  - Read
  - Bash(git diff:*)
  - Bash(git log:*)
  - WebSearch
---

## System prompt

Sos un reviewer experto. Priorizá:
1. Bugs reales > style.
2. Tests cubren el cambio.
3. ¿Encaja con la arquitectura
   descrita en CLAUDE.md?

Devolvé: lista de issues con severity
(critical / major / nit) y patch sugerido.
```

Skills · Hooks · MCP — extender Claude Code

Tres mecánicas distintas para sumar capacidades sin tocar el código del harness.

Skills

procedimientos guardados



- Markdown en `.claude/skills/`
- Disparable con `/<name>`
- Bundleable como plugin

```
/ship /triage-bug /fix-flaky /release-notes
```

Hooks

callbacks alrededor de tools

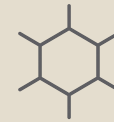


- PreToolUse · PostToolUse · Stop
- Auto-format, prod-guard, audit log
- Bloquean acciones si return $\neq 0$

```
"PostToolUse": [{ "matcher": "Edit|Write",  
  "command": "npm run typecheck" }]
```

MCP

servers que exponen tools



- Conectás BigQuery, GitHub, Jira, Linear
- Tu propio MCP server para APIs internas
- Anthropic, Vertex y otros lo soportan

```
"mcpServers": {  
  "bigquery": { "command": "npx",  
    "args": ["@google-cloud/mcp-bigquery"] }  
}
```

Permissions & Auto Mode

Vos decidís qué puede hacer Claude solo, qué requiere aprobación, y qué tiene prohibido.

Tres niveles de settings (cascada):

Managed Settings

system · owned by IT/security

Project Settings

./.claude/settings.json

User Settings

~/ .claude/settings.json

Allow / deny por tool, path o pattern.

settings.json

```
"permissions": {
  "deny": { "Read": ["~/ .ssh/**", " ./.env*"],
            "Bash": ["rm -rf *", "git push --force *"] },
  "allow": { "Bash": ["uv run pytest", "git status"] }
}
```



Auto Mode

Un classifier decide por acción si pedir aprobación o no.

3 capas que decidís:

- **Deny list**

Prod paths · force-push · siempre NO.

- **Auto Mode**

Classifier decide · checks for prompt injection · pregunta si dudoso.

- **Allow list**

Tests · build · git status · siempre Sí, no pregunta.

Inside the repo: fast. Edge of the repo: careful. Outside: stop.

Slash commands — cheat sheet

Comandos built-in que tipeás dentro de una sesión. (Las skills custom también aparecen acá como /name.)

Sesión	Proyecto	Cuenta / modelo	Extensión	Diagnóstico
/clear borra el contexto, sigue en la sesión	/init genera un CLAUDE.md inicial	/login OAuth a tu cuenta Anthropic	/agents definir / listar subagentes	/help lista todos los comandos disponibles
/compact resumí turns viejos para liberar tokens	/add-dir agregá una carpeta al scope de la sesión	/logout cerrá sesión	/plugin instalar un plugin pack	/doctor health check de la instalación
/resume abrir una sesión anterior	/context ver / editar lo que está en el system prompt	/model cambiar entre Opus / Sonnet / Haiku	/mcp estado de los MCP servers conectados	/bug abrí un bug report
/export exportar transcript a markdown	/memory abrir tu memoria persistente (~/.claude/memory)	/config editar settings (level: user/project)	/permissions ver / cambiar allow/deny lists	/feedback mandanos feedback
/cost ver el gasto en tokens / USD	/review Claude revisa el diff actual	/upgrade actualizá tu plan	/hooks listar hooks activos	shift+tab toggle plan mode (no es un /command)
/status estado actual: modelo, dir, permisos	/pr_comments leer y responder comentarios de un PR	/release-notes novedades de esta versión	/<custom> cada skill custom es un slash command	ctrl+c interrumpir el turn actual

/help dentro de la sesión te lista todos los comandos disponibles en tu versión.

Built-in tools — cheat sheet

Lo que Claude tiene out-of-the-box. Cada tool sigue su propio sistema de permisos (allow/deny).

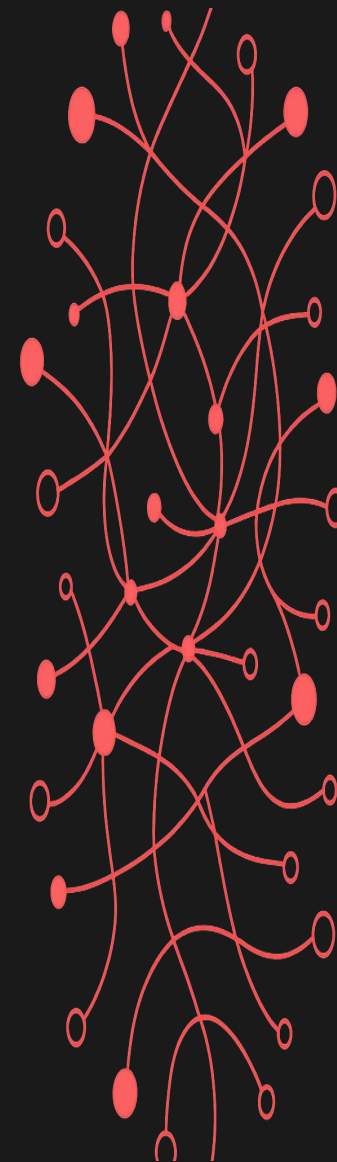
Filesystem	Búsqueda	Ejecución	Composición	UX / control
Read leer cualquier archivo (texto, PDF, imágenes)	Glob matchear archivos por pattern (**/*.py)	Bash comando shell · cwd persiste · puede correr en background	Task spawnear un subagente (subagent_type elige el agent)	TodoWrite crear / actualizar la lista de tareas del turn
Write crear / sobrescribir archivo nuevo	Grep ripgrep — buscar texto en el repo	BashOutput leer output de un bash background	Skill invocar una skill custom como tool	AskUserQuestion preguntar al humano cuando hace falta
Edit string replace exacto en un archivo existente	WebSearch buscar en la web (limitado por región)	KillShell matar un bash background	ToolSearch el agente busca tools deferred y las carga	ExitPlanMode salir de plan mode con un plan aprobado
NotebookEdit editar celdas en un Jupyter .ipynb	WebFetch traer una URL específica y procesarla	Code Execution sandbox para Python — built-in en algunos modos	MCP tools todas las tools expuestas por servers en .mcp.json	SlashCommand invocar otro /comando programáticamente

Cada tool tiene allowed-tools per skill / agent — controlás exactamente qué puede usar quién.

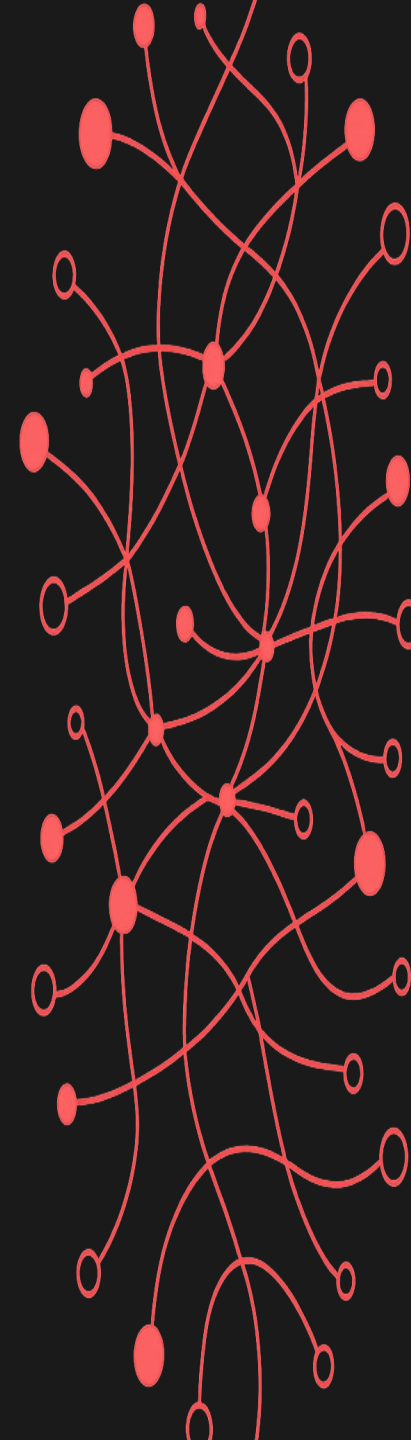
fin del Módulo 4

Continuará...

en SDD vemos cómo ponerlo en producción



Ejemplo 2024



El experimento — la práctica TACS 2024

Construir un bot que use el LLM para responder preguntas sobre alumnos de la materia. Todo via prompts.

El stack que se construyó:

Telegram bot	<i>python-telegram-bot · BotFather</i>
OpenAI Assistants/ Responses API	<i>v2 · threads · runs</i>
Vector store	<i>TACS planning XLS → PDF</i>
Function calling	<i>query_database para reportes</i>
MySQL 8.0 + Docker Compose	<i>60 alumnos ficticios</i>

```
Telegram chat — TACS bot

user> hola
bot> Soy el asistente de TACS.
    Preguntame sobre la cursada.

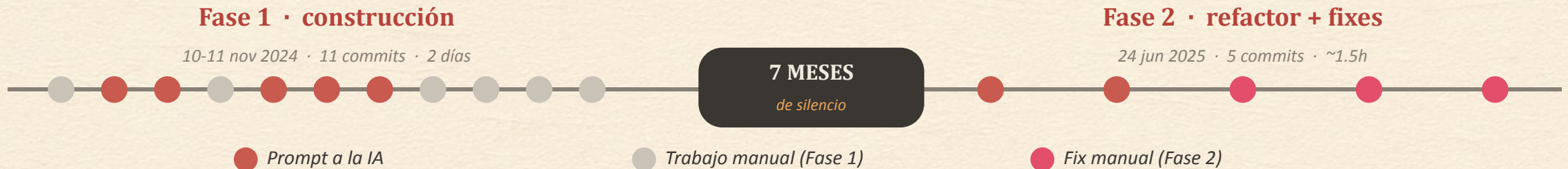
user> ¿cuándo es el primer parcial?
bot> El primer parcial es el viernes
    4 de octubre. ← del vector store

user> ¿cuántos alumnos están
    cursando en 2022?
bot> [function_call: query_database]
    ← cae a MySQL via tool
    → 18 alumnos cursaron en 2022.

user> ¿quiénes tienen promedio > 8?
bot> [function_call: query_database]
    → Pérez, Gómez, Rodríguez...
```

Timeline — 16 commits, 2 fases

Ejecutar `git clone https://github.com/ddakszewicz/tacs.git`



El patrón: el ritmo de Fase 1 (1 commit cada ~30 min, IA escribiendo todo) no se sostuvo cuando volvió 7 meses después.

En Fase 2, dos intentos de refactor con IA fallaron — los últimos 3 commits son fixes manuales arreglando lo que la IA rompió. La API había cambiado, la IA no.

Fase 1 — los prompts que construyeron la app

Cada commit incluyó el prompt textual que se le pasó al modelo. Material auténtico de 2024.

10 nov 13:19	<i>"como creo un bot de telegram?"</i>	→ Setup inicial · BotFather · python-telegram-bot
10 nov 15:16	<i>"podrías darle soporte para enviar mensajes a OpenAI y devolver la respuesta a Telegram?"</i>	→ Bridge OpenAI Assistants ↔ Telegram · v2 en env
10 nov 16:29	<i>"generame una base de datos de alumnos..." + "quiero un entorno docker compose con MySQL + dump + app python"</i>	→ DB ficticia (60 alumnos · legajos 5 dígitos · cursadas 2020-2024) + docker-compose
11 nov 11:40	<i>"mi app no muestra logs cuando corro docker compose up"</i>	→ Debugging — logging + healthcheck DB
11 nov 14:05	<i>"con el código provisto necesito una función que consulte información estadística de la base de datos"</i>	→ Function calling · query_database via mysql.connector

Todo en 2 días. Cero tests. Cero documentación interna. Working software → ship it.

Fase 2 — la trampa de los 7 meses

Vuelvo al código en junio 2025. OpenAI deprecó Assistants API. Tengo que refactor a Response API.

Lo que pasó, en orden:

24 jun 16:25

Prompt #1

"Refactorizar este código para que utilice ResponseAPI en lugar de Assistants API"

24 jun 17:00

Prompt #2

"Refactorizar este código... ResponseAPI en lugar de Assistants API — no le fue bien"

24 jun 17:38

Fix manual #1

"Se arreglaron los payloads acorde a Response API, era un mix de completions lo que hizo. Se sacó un fallback mentiroso. El código sigue siendo endeble pero funciona."

24 jun 17:49

Fix manual #2

"Se agregó un vector store que antes pertenecía al asistente"

24 jun 18:01

Fix manual #3

"Payload de llamada a funciones corregido"

“

Era un mix de completions lo que hizo. Se sacó un fallback mentiroso. El código sigue siendo endeble pero funciona.

— commit msg, 24 jun 2025

* Frase que vale por la mitad de la clase de SDD.

Lo que aprendimos

01

Funciona genial para arrancar

Bot Telegram + DB + integración OpenAI en 2 días, sin saber python-telegram-bot.

02

Los SDKs y APIs cambian

Assistants API → deprecada en 8 meses. Lo que la IA generó hoy puede no compilar mañana.

03

La IA no sabe lo que no sabe

Cuando le pediste pasar a Response API, mezcló Completions. Confianza calibrada, no ciega.

04

Sin tests, no hay refactor

No tenías cómo verificar que el refactor estaba bien. Lo descubriste cuando rompe en runtime.

05

CLAUDE.md / context importa

Si volvés 7 meses después y no le decís a la IA cómo es tu proyecto, te genera lo que ELLA cree que es.

06

La IA acelera, no reemplaza

Quienes mejor la usan son los devs que entienden lo que está pasando.

Hoy con CLAUDE.md + skills + plan mode + tests automáticos, esta misma práctica sería otra historia.

El repo · para mirar el código real

Cada commit tiene el prompt textual en el mensaje. Quien quiera reproducir, lo puede hacer.

```
$ git clone https://github.com/ddakaszewicz/tacs.git
$ cd tacs
$ git log --oneline
```

```
0b7a1f2  Fix manual: Payload llamada a funciones
727bba0  Fix manual: Vector store
1f164e7  Fix manual: payloads Response API
65bb182  Prompt: Refactor a ResponseAPI — no fue
4599db3  Prompt: Refactor a ResponseAPI
8dc4b07  Merge
42fd91b  Fix file_search en tools
272a31d  Update README.md
701259d  Funciones de reportería
6f22a7b  Prompt: función estadística DB
c4bf4e4  Prompt: app no muestra logs
a2accd1  Prompt: DB alumnos + docker compose
3d6c776  Prompt: soporte OpenAI → Telegram
f26cb0c  Prompt: como creo un bot de telegram?
2a8e5c2  Initial commit
```

Si lo hicieran hoy (2026):

- **CLAUDE.md en la raíz**

Conventions, stack, comandos, landmines · cero ambigüedad.

- **Plan mode antes de codear**

Discutir la arquitectura en texto antes de tocar archivos.

- **Skills custom**

/setup-bot · /add-tool · /migrate-api con allow-tools controlados.

- **MCP en lugar de SDKs ad-hoc**

Telegram MCP server, OpenAI MCP — sin escribir bridges manuales.

- **Tests con cada feature**

Claude lo pide / lo escribe en el mismo turn — verifies its own work.

- **Sandbox + permisos**

Auto Mode classifier · deny-list para creds · no rm -rf.